Optimizing Search-Based Unit Test Generation with Large Language Models: An Empirical Study

Danni Xiao
201220199@smail.nju.edu.cn
State Key Laboratory for Novel Software Technology,
Nanjing University
Nanjing, China

Yimeng Guo
ymguo@smail.nju.edu.cn
State Key Laboratory for Novel Software Technology,
Nanjing University
Nanjing, China

Yanhui Li
yanhuili@nju.edu.cn
State Key Laboratory for Novel Software Technology,
Nanjing University
Nanjing, China

Lin Chen*
lchen@nju.edu.cn
State Key Laboratory for Novel Software Technology,
Nanjing University
Nanjing, China

ABSTRACT

Search-based unit test generation methods have been considered effective and widely applied, and Large Language Models (LLMs) have also demonstrated their powerful generation ability. Therefore, some scholars have proposed using LLMs to enhance search-based unit test generation methods and have preliminarily confirmed that LLMs can help alleviate the problem of test coverage plateaus. However, it is still unclear when and how LLMs should intervene in the time-consuming test generation process. This paper explores the application of LLMs at various stages of search-based test generation (SBTG) (including the initial stage, the test generation period, and the test coverage plateaus), as well as strategies for controlling the frequency of LLM intervention. A comprehensive empirical study was conducted on 486 Python benchmark modules from 27 projects. The experimental results show that 1) LLM intervention has a positive effect at any stage, whether to improve coverage over a fixed period or to reduce the time to reach a specific coverage; 2) a reasonable intervention frequency is crucial for LLMs to have a positive effect on SBTG. This work can better help understand when and how LLMs should be applied in SBTG and provide valuable suggestions for developers in practice.

CCS CONCEPTS

• **Software and its engineering → Search-based software engineering**.

KEYWORDS

Unit Test, Search-based Testing, Large Language Model

*Corresponding author

Internetware 2024, July 24–26, 2024, Macau, China

ACM ISBN 979-8-4007-0705-6/24/07
https://doi.org/10.1145/3671016.3674813

ACM Reference Format:
Danni Xiao, Yimeng Guo, Yanhui Li, and Lin Chen. 2024. Optimizing Search-Based Unit Test Generation with Large Language Models: An Empirical Study. In *15th Asia-Pacific Symposium on Internetware (Internetware 2024), July 24–26, 2024, Macau, China.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3671016.3674813

1 INTRODUCTION

Unit test generation is crucial in software development as it ensures the reliability and correctness of code by automatically generating tests to evaluate individual units of code in isolation [32]. These tests verify that each unit functions as expected, helping to catch bugs early in the development process and facilitating code maintenance and refactoring. By automating the process of generating tests, developers can increase test coverage and reduce the burden of manual testing [27], ultimately improving the overall quality of the software. Additionally, when serving as documentation, unit tests can provide insight into the intended behavior of the code and help developers understand its functionality. This proactive approach to testing ensures the software quality and allows for more confident and efficient software delivery.

Search-based test generation (SBTG) is a vital method in unit testing that has gained significant attention due to its effectiveness in enhancing testing quality [3, 13, 14, 17, 29, 31]. This method employs search algorithms, such as genetic algorithms [6, 18, 22] or simulated annealing [36], to systematically explore the space of possible test cases and identify inputs that satisfy specific criteria, such as achieving high code coverage or revealing faults. By leveraging optimization techniques, SBTG can efficiently produce comprehensive test suites that target critical parts of the code, leading to improved testing quality. This method enables developers to uncover edge cases and corner scenarios that might not be apparent through manual testing, thereby enhancing the robustness and reliability of the software under test.

While SBTG methods offer significant benefits, they also have certain drawbacks and face challenges [1, 2]. One prominent challenge is the computational overhead associated with exploring large search spaces, especially in complex software systems with numerous input parameters and dependencies. This can lead to long execution times and scalability issues, making it impractical

for real-time or resource-constrained environments. Additionally, search-based methods heavily rely on heuristics and optimization algorithms, which may not always guarantee the discovery of optimal or near-optimal solutions. Consequently, there is a risk of generating redundant or irrelevant test cases that do not effectively improve testing coverage or reveal faults. Overall, while search-based methods offer powerful automated testing capabilities, addressing these challenges is crucial to realizing their full potential in software testing practices.

Some researchers have explored the application of Large Language Models (LLMs) for SBTG [10, 23, 25, 26, 33, 39, 40], particularly to address the issue of stagnant test coverage. By leveraging the natural language generation capabilities of these models, developers can specify testing objectives and constraints more intuitively and expressively. LLMs can assist in generating diverse and contextually relevant test cases by understanding the semantics of the code under test and its associated requirements [10, 23, 25, 26, 33, 39, 40]. This makes it promising for overcoming the limitations of traditional search-based methods, as LLMs enable more sophisticated exploration of the test space and the generation of test cases tailored to specific testing criteria [12, 34, 37]. By harnessing the capabilities of LLMs, developers may enhance test coverage and effectiveness, leading to improved software quality and reliability.

However, current research still lacks an in-depth exploration of LLM intervention in SBTG at different stages and in different manners. For example, achieving higher test coverage in a given execution time and obtaining relatively satisfactory coverage in a short time are possible test requirements for developers in practice.

Therefore, this paper conducts a comprehensive empirical study on analyzing various strategies for LLMs to intervene in SBTG. We aim to explore when and how LLMs should be applied to help SBTG improve test coverage effectively and efficiently. The following research questions are to be answered:

- **RQ1 (Stage of Intervention):** When should LLM intervention be applied to optimize test coverage effectively?
- **RQ2 (Manner of Intervention):** In what manner does LLM intervention contribute more effectively to test coverage improvement?

We applied different LLM intervention strategies on 486 Python benchmark modules and analyzed the results of the experiments. The strategies include various stages and manners that apply LLMs to SBTG. The stages include *initializing population*, *iterative updating population*, and *coverage plateaus*. The manners we use mainly *control the frequency* of LLM intervention and *dynamic monitor* the intervention. The results show that LLMs can improve test coverage of SBTG but excessive LLM intervention may reduce the effectiveness. To obtain higher test coverage within a given time, the best strategy is to apply LLM intervention when SBTG generates the initial test population, set the initial probability to a larger value (such as 1.0), and monitor it dynamically.

In summary, the main contributions of this paper are:

- A comprehensive empirical study was conducted on 486 Python benchmark modules from 27 projects, to explore when and how LLMs should be applied in the time-consuming SBTG process.
- The study yields some interesting and valuable findings, particularly confirming that LLM intervention can positively impact SBTG at all stages. Furthermore, the findings about the necessity of balancing the roles and timing of LLMs and SBTG, which were not well understood before, are particularly noteworthy.

The remainder of this paper is organized as follows. Section 2 introduces the background and our motivation. Section 3 describes the experimental setup of our study. In section 4, we show and analyze the experimental results. Section 5 discusses the implications and limitations of this study. Section 6 presents related work. Section 7 draws our conclusion.

2 BACKGROUND AND MOTIVATION

In this section, we introduce the background of search algorithms and the motivation for applying LLM intervention.

2.1 Search Algorithm

The search algorithm is the key factor for search-based test generation (SBTG) to efficiently explore the test input space and generate the test suite that best meets the given requirements. Common algorithms in SBTG include genetic algorithm, hill climbing algorithm, simulated annealing, etc. We select the genetic algorithm, one of the most widely used search algorithms in the field of SBTG, for our empirical research.

Genetic algorithm draws inspiration from the biological process of natural selection and genetics, imitates the genetic evolution process of species populations in nature, and allows a large number of candidate solutions for optimization problems to be continuously updated and iterated, gradually approaching the optimal solution set. It is very effective for solving optimization problems with large and complex solution spaces, which is why it is suitable for SBTG. As shown in Figure 1, the genetic algorithm mainly includes the following phases.

(1) **Initialize population.** The genetic algorithm starts by randomly generating a population of N test cases. The initial population will be evaluated and ranked according to the fitness function, which quantifies the gap between each solution and the target.
(2) **Iterative update population.**
   (a) *Generate offspring.* First, select the test cases with higher fitness scores in the current population as the parents of the next generation. Then, crossover and mutation operators are applied to the parents to obtain offspring. Selection methods include roulette wheel selection, tournament selection, and ranking selection. The crossover operator imitates the reproduction and recombination of organisms, allowing parents to exchange part of their genetic information. For test cases, this means exchanging part of the code to obtain new test cases. The mutation operator randomly changes part of the genetic information of the offspring with a certain probability. For test cases, it usually involves operations such as adding, deleting, and modifying code statements.
   (b) *Evaluate the offspring and the population as a whole.* After obtaining the offspring population, evaluate and rank the

offspring and the population as a whole according to the fitness function.

(c) *Replace.* Select the top N rankings among offspring and the population according to the ranking, that is, replace part or all of the existing population with the offspring to form a new generation of population.

(3) **Coverage plateau and Terminate.** Repeat the process of selection, crossover, mutation, evaluation, and replacement until the termination condition is met, such as the test time is exhausted or the code coverage reaches 100%.

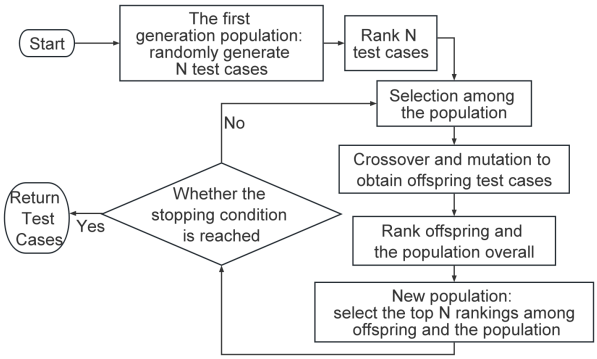


Figure 1: Genetic Algorithm

2.2 Motivation: LLM Intervention

Previous work [24] has shown that LLMs may help SBGT overcome coverage plateaus. Coverage plateaus occurs when SBGT has been running for an extended period without further improving test coverage. From the aforementioned SBTG process, it can be seen that coverage plateau is only the last stage of the SBTG process. SBTG is multi-stage and time-consuming, which suggests that *LLMs can intervene in test generation at various stages and in different manners*. This insight motivates us to conduct a comprehensive study to understand **when** and **how** LLMs could intervene and impact the SBGT process.

2.2.1 *When should LLMs be applied?*

There are some stages of SBTG that LLMs can be applied.

Stage of Initializing Population. Involving the LLM at this stage can alter the composition of the initial population. At this point, all coverage targets have a coverage of zero and there is no order of priority among them. Therefore, the LLM can randomly select a target from all the objects under test to generate test cases. By exploring the impact of incorporating LLM-generated test cases into the initial population on code coverage, we aim to determine whether early intervention of the LLM is effective, including whether it affects real-time coverage and whether it improves the final coverage.

Stage of Iterative Updating Population. At this stage, when we involve the LLM, we direct its generation targets towards the tested objects with relatively low coverage. The generated test cases are then used as parents for the crossover operator and are screened along with test cases obtained through crossover and mutation into the next generation population. This approach aims to determine whether the LLM can generate values in the input space that MOSA might have difficulty exploring. If so, we seek to understand the extent to which the inclusion of LLM-generated test cases in the iterative loop of MOSA will ultimately improve coverage.

Stage of Test Coverage Plateau. The test coverage plateau stage is a specific period during the iterative update of the population. The test coverage plateau phenomenon refers to the situation where the coverage of the new generation in MOSA is the same as that of the previous generation. This means that the search of the input space has stagnated, preventing further exploration of new input spaces and thus hindering any increase in coverage. We involve the LLM during the test coverage plateau to study whether and to what extent the LLM can alleviate this issue.

2.2.2 *How should LLMs be applied?*

There are some ways to control the LLM intervention.

Random Intervention. During both the initialing population stage and the iterative updating population stage, the LLM can be randomly applied with a probability p. If a randomly generated decimal is less than or equal to p, the LLM will be applied; otherwise, the process remains unchanged.

Controlling the Length of Coverage Plateaus. As SBGT incorporates randomness, it is difficult to predict whether continuing the search will improve coverage once it stagnates. Therefore, a maximum duration threshold can be set for coverage plateau. If the test coverage plateau period exceeds this threshold, the LLM will be applied.

Dynamic Monitor. Since both SBGT and LLM queries are time-consuming activities, it is essential to maximize their effectiveness within a limited time. This means that SBGT should not run for extended periods without improving test coverage, nor should LLMs be overly involved if it does not contribute to coverage improvement. Dynamic monitoring of the performance of both SBGT and LLMs is necessary, which can enable timely adjustments to their execution. Specifically, we make the cooperation between SBTG and LLMs more effective by dynamically adjusting the above two control factors of LLM intervention.

3 EXPERIMENTAL SETUP

In this study, we chose Pynguin[1] as the basic framework for the experiment. Pynguin is a SBTG framework based on Python that provides the implementation of the MOSA (**M**any-**O**bjective **S**orting **A**lgorithm). As for the LLM, Code Llama of Llama2 was selected. We modified and supplemented the code based on Pynguin 0.19.0, and completed the code implementation of our study.

Regarding the details of using LLMs to generate test cases, the key points are the prompt engineering and post-processing steps. As for the prompt, we used the source code of the module under test and the function header of its test function. Referring to the approach of CODAMOSA [24], we rewrote the output of Code Llama into test cases that conformed to the Pynguin format.

[1] https://www.pynguin.eu/

3.1 Experimental Data

We use the dataset of CODAMOSA, which includes 486 benchmark modules in 27 projects from Pynguin and BugsInPy [38]. Considering that it would take too long to execute all 486 modules several times using various strategies, we preliminarily filtered the 486 modules on the premise of ensuring the validity of the experimental results. We referred to the experimental results of CODAMOSA and retained all modules that had achieved more than 15% coverage improvement after LLM intervention, and randomly selected 20 remaining modules to form the final dataset for our complete experiment. The data selection is based on two considerations. First, to ensure there are enough samples whose coverage can be improved so that we can further observe the effects of LLM intervention in SBGT at different stages and in different manners. Second, even for samples that did not improve coverage in the previous experiment, we also hope to explore the potential effects of LLM intervention before test coverage stagnates.

3.2 Experimental Parameters

The parameters in our experiment are related to Pynguin configuration and LLM intervention strategies.

3.2.1 *Pynguin-related Parameters.*

For Pynguin-related parameter configuration, we chose Pynguin's default values. The test case population size is the default value of 50, and the search time is the default value of 10 minutes. For the parameter configuration of the LLM, we referred to the experimental results of CODAMOSA to maximize the effectiveness of LLM intervention. The temperature parameter of the LLM is set to 0.8 and the maximum number of LLM queries per intervention is 10 times.

3.2.2 *Experimental Parameters of Intervention Strategies.*

As shown in Section 2, the LLM can intervene in SBTG at different stages in different manners. We first introduce all experimental parameters and Algorithm 1 illustrates the complete process of applying the LLM to MOSA. Different experimental settings in the algorithm can be obtained by setting different parameter combinations, which are shown in Table 1 and Table 2.

***I**nitializing Population & **R**andom Intervention* (**MOSA+LLM-ir**): For the selection of random probability, to observe the effect of different probabilities on LLM intervention and coverage during the experiment, we selected 4 probability values from 0 to 1, which are 0.05, 0.1, 0.5, and 1.0. (lines 3 to 4 of Algorithm 1)

***I**nitializing Population & **R**andom Intervention & **D**ynamic **M**onitor* (**MOSA+LLM-irdm**): In experiments with a certain initial probability, we chose an initial probability of 1.0. The dynamic monitoring method is to decide whether the probability of LLM intervention needs to be halved based on whether the LLM returns a test case. (lines 3 to 7 of Algorithm 1)

*I**t**erative Updating Population & **R**andom Intervention* (**MOSA+LLM-tr**): Similar to MOSA+LLM-ir, we selected 4 probability values of 0.005, 0.1, 0.5, and 1.0. (lines 15 to 17 of Algorithm 1)

*I**t**erative Updating Population & **R**andom Intervention & **D**ynamic **M**onitor* (**MOSA+LLM-trdm**): Similarly, the experiments are with the certain initial probability of 1.0. After each LLM intervention, we decide whether the probability of LLM intervention needs to be halved based on whether the new generation group has test cases

Algorithm 1 Many-Objective Sorting Algorithm with the LLM

Input: *module* to test, population size N, search time T, stopping condition C, probability of the LLM randomly intervening when generating the initial population p_{ir}, probability of the LLM randomly intervening during testing p_{tr}, maximum stagnation length $maxStallLen$
Output: archive of optimised test cases $archive$

1: $t \leftarrow 0$
2: **for** $i = 1 \rightarrow N$ **do**
3: **if** $GET-RANDOM() <= p_{ir}$ **then** ▷ MOSA+LLM-ir/irdm
4: $testCase_i \leftarrow GENERATE-RANDOM-TESTCASE-WITH-LLM()$
5: **if** $testCase_i$ *is None* **then** ▷ MOSA+LLM-irdm
6: $p_{ir} \leftarrow p_{ir} / 2$
7: $testCase_i \leftarrow GENERATE-RANDOM-TESTCASE()$
8: **else**
9: $testCase_i \leftarrow GENERATE-RANDOM-TESTCASE()$
10: $P_t \leftarrow P_t \cup \{testCase_i\}$
11: $UPDATE-ARCHIVE(archive, P_t)$
12: $PERFORM-FITNESS-EVALUATION(P_t)$
13: $stallLen \leftarrow 0$ ▷ MOSA+LLM-tl/tldm
14: **while** $\neg C$ **do**
15: **if** $GET-RANDOM() <= p_{tr}$ **then** ▷ MOSA+LLM-tr/trdm
16: $wasIntervened \leftarrow True$
17: $P_o \leftarrow GENERATE-OFFSPRING-WITH-LLM(P_t)$
18: **else**
19: $wasIntervened \leftarrow False$
20: $P_o \leftarrow GENERATE-OFFSPRING(P_t)$
21: $coverageBefore \leftarrow COVERAGE(archive)$ ▷ MOSA+LLM-tl/tldm
22: **if** $stallLen > maxStallLen$ **then** ▷ MOSA+LLM-tl/tldm
23: $wasIntervened \leftarrow True$
24: $P_o \leftarrow GENERATE-OFFSPRING-WITH-LLM(P_t)$
25: $stallLen \leftarrow 0$
26: **else**
27: $wasIntervened \leftarrow False$
28: $P_o \leftarrow GENERATE-OFFSPRING(P_t)$
29: $U_t \leftarrow P_t \cup P_o$
30: $Fronts \leftarrow PREFERENCE-SORTING(U_t)$
31: $P_{t+1} \leftarrow \{\}$
32: $i \leftarrow 0$
33: **while** $|P_{t+1}| + |Front_i| < N$ **do**
34: $CALCULATE-CROWDING-DISTANCE(Front_i)$
35: $P_{t+1} \leftarrow P_t \cup Front_i$
36: $i \leftarrow i + 1$
37: $DISTANCE-CROWDING-SORT(Front_i)$
38: $P_{t+1} \leftarrow P_{t+1} \cup Front_i$ *with size* $N - |P_{t+1}|$
39: **if** $wasIntervened$ *and* $P_{t+1} \subseteq P_t$ **then** ▷ MOSA+LLM-trdm
40: $p_{tr} \leftarrow p_{tr} / 2$
41: **if** $wasIntervened$ *and* $P_{t+1} \subseteq P_t$ **then** ▷ MOSA+LLM-tldm
42: $maxStallLen \leftarrow maxStallLen * 2$
43: **if** $COVERAGE(archive) = coverageBefore$ **then** ▷ MOSA+LLM-tl/tldm
44: $stallLen \leftarrow stallLen + 1$
45: **else**
46: $stallLen \leftarrow 0$
47: $UPDATE-ARCHIVE(archive, P_{t+1})$
48: $t \leftarrow t + 1$
49: **return** $archive$

Table 1: Intervention Strategies of LLMs

Strategy	Stage	Manner
MOSA+LLM-ir (ir)	Initializing population	Random Intervention
MOSA+LLM-irdm (irdm)	Initializing population	Random Intervention + Dynamic monitor
MOSA+LLM-tr (tr)	Iterative Updating Population	Random Intervention
MOSA+LLM-trdm (trdm)	Iterative Updating Population	Random Intervention + Dynamic monitor
MOSA+LLM-tl (tl)	Test Coverage Plateaus	Controlling the Length of Coverage Plateaus
MOSA+LLM-tldm (tldm)	Test Coverage Plateaus	Controlling the Length of Coverage Plateaus + Dynamic monitor

Table 2: Experimental Parameters of Intervention Strategies

strategy	parameter	values
MOSA+LLM-ir (ir)	probability	0.05 0.1 0.5 1.0
MOSA+LLM-irdm (irdm)	probability	1.0
MOSA+LLM-tr(tr)	probability	0.005 0.1 0.5 1.0
MOSA+LLM-trdm (trdm)	probability	1.0
MOSA+LLM-tl (tl)	max stagnation length	5 25 50 200 800
MOSA+LLM-tldm (tldm)	max stagnation length	5

that are not in the previous generation group. (lines 15 to 17 and 39 to 40 of Algorithm 1)

***T**est Coverage Plateau & Controlling the **L**ength of Coverage Plateau* (**MOSA+LLM-tl**): Based on the observations of preliminary experiments, we selected 5 values of maximum stagnation length as the parameters of the experiment, which are 5, 25, 50, 200, and 800. (lines 21 to 25 and 43 to 46 of Algorithm 1)

***T**est Coverage Plateau & Controlling the **L**ength of Coverage Plateau & **D**ynamic **M**onitor* (**MOSA+LLM-tldm**): In experiments with the initial maximum stagnation length of 5, after each LLM intervention, the maximum stagnation length will double if the new generation group has test cases that are not in the previous generation group. (lines 21 to 25 and 41 to 46 of Algorithm 1)

We repeated the experiment three times for each LLM intervention strategy mentioned above on each Python module.

3.3 Evaluation Metrics

We use coverage as a metric to evaluate the execution effectiveness of MOSA and MOSA+LLM. In addition to line coverage, we also consider branch coverage. Branch coverage is defined as the number of covered branches in the tested object, which is the number of executed branches, divided by the number of all branches.

To answer RQ1 and RQ2, we not only get the final coverage but also randomly record the coverage value during the execution process, which is referred to as real-time coverage.

4 STUDY RESULTS

Due to page limitation, only representative experimental results are shown in this paper. In Figure 2, Figure 3, and Figure4, we use scatter plots to show the final coverage for each strategy comparing all MOSA+LLMs with MOSA. In Figure 5, Figure 6, and Figure 7, we draw curve graphs to show the real-time coverage difference between MOSA+LLMs and MOSA over time.

4.1 RQ1: Stage of Intervention

In RQ1, we analyze the effect of LLM intervention at different stages on the final coverage and real-time coverage.

4.1.1 Final Coverage.

Figure 2, Figure 3, and Figure 4 show that various LLM intervention strategies can improve the final coverage of MOSA on multiple Python modules. At the same time, the final test coverage of a few Python modules is lower than that of MOSA under most strategies.

> **Finding 1:** MOSA+LLM-ir/irdm, MOSA+LLM-tl/tldm and MOSA+LLM-tr/trdm have the potential to greatly improve the final coverage, but there is also the possibility that the final coverage cannot be improved or even decreases.

As for the number of modules whose coverage is improved, there is not much difference among MOSA+LLM-ir/irdm, MOSA+LLM-tl/tldm, and MOSA+LLM-tr/trdm. But the number of modules whose coverage is reduced by MOSA+LLM-ir/irdm and MOSA+LLM-tr/trdm is greater than that of MOSA+LLM-tl/tldm.

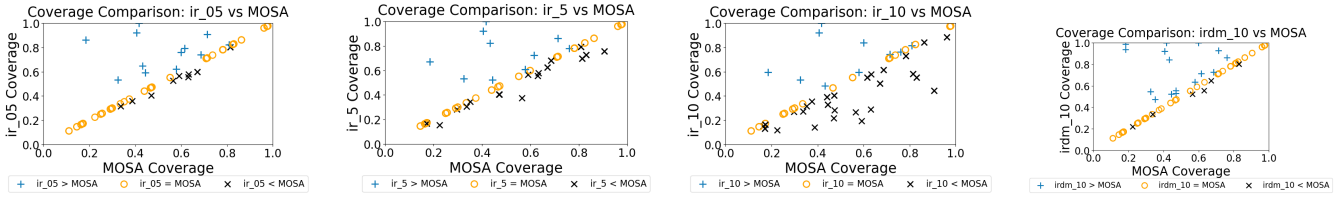
As for the average improvement, MOSA +LLM-ir/irdm, MOSA+LLM-tl/tldm, and MOSA+LLM-tr/trdm decrease in turn. From the perspective of the degree of coverage reduction, MOSA+LLM-ir/irdm, and MOSA+LLM-tr/trdm are greater than MOSA+LLM-tl/tldm.

> **Finding 2:** In general, MOSA+LLM-ir/irdm has the best average improvement while MOSA+LLM-tr/trdm has the worst, and MOSA+LLM-tl/tldm is in between.

4.1.2 Real-time coverage. From Figure 5, Figure 6, and Figure 7, we can see that a large number of curves in each figure are above the line with the vertical axis of 0, meaning that the strategy improves the coverage of MOSA. In the beginning, the coverage of MOSA+LLM-ir/irdm is often higher than that of MOSA. There are Some curves of MOSA+LLM-ir/irdm and MOSA+LLM-tr/trdm are below the line with the vertical axis of 0, especially MOSA+LLM-ir/irdm. This means that the average coverage of MOSA decreases when applying these strategies. But similar cases are rare in MOSA+LLM-tl/tldm.

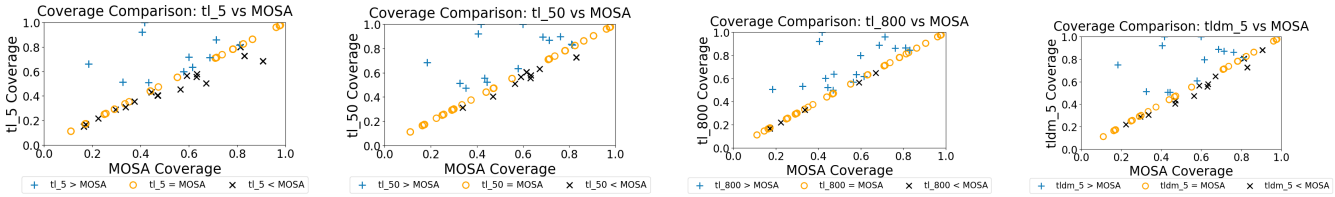
> **Finding 3:** LLM intervention at three stages has the potential to improve the real-time coverage. Especially, when it intervenes at the stage of generating the initial population, the coverage may be greatly improved from the beginning.

Answer to RQ1. LLM intervention has a positive effect at all stages, whether to improve coverage over a fixed period or to reduce the time to reach a specific coverage. The average effect is



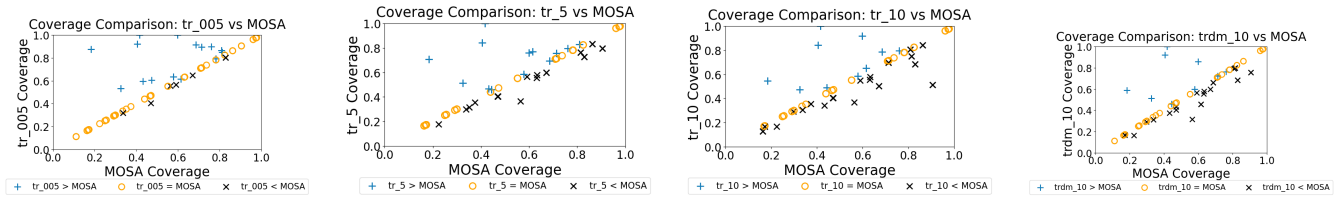
(a) $MOSA + LLM - ir\_05$ **(b)** $MOSA + LLM - ir\_5$ **(c)** $MOSA + LLM - ir\_10$ **(d)** $MOSA + LLM - irdm\_10$

Figure 2: Average coverage achieved by MOSA+LLM-ir vs. MOSA at the end of search time.



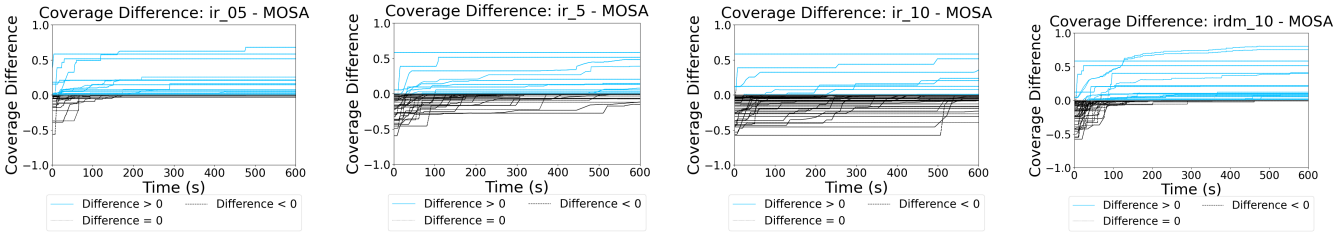
(a) $MOSA + LLM - tl\_5$ **(b)** $MOSA + LLM - tl\_50$ **(c)** $MOSA + LLM - tl\_800$ **(d)** $MOSA + LLM - tldm\_5$

Figure 3: Average coverage achieved by MOSA+LLM-tl vs. MOSA at the end of search time.



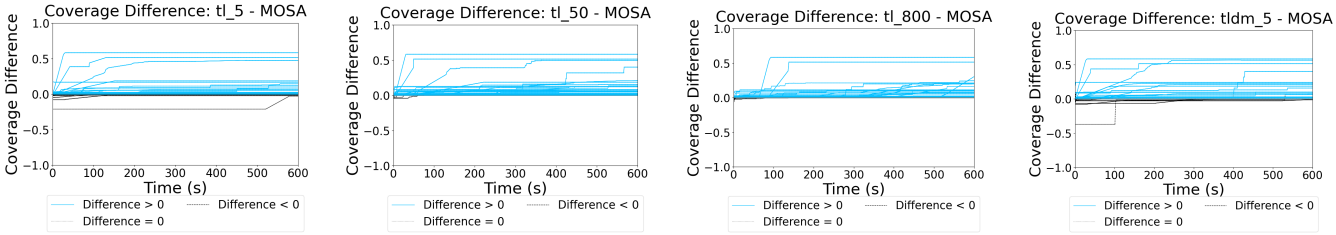
(a) $MOSA + LLM - tr\_005$ **(b)** $MOSA + LLM - tr\_5$ **(c)** $MOSA + LLM - tr\_10$ **(d)** $MOSA + LLM - trdm\_10$

Figure 4: Average coverage achieved by MOSA+LLM-tr vs. MOSA at the end of search time.



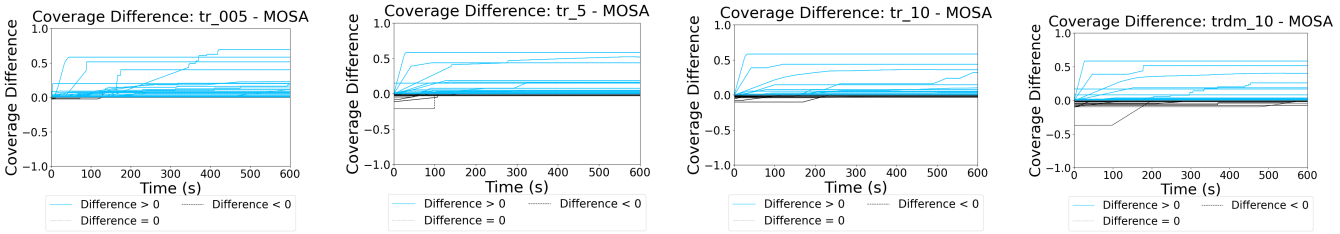
(a) $MOSA + LLM - ir\_05$ **(b)** $MOSA + LLM - ir\_5$ **(c)** $MOSA + LLM - ir\_10$ **(d)** $MOSA + LLM - irdm\_10$

Figure 5: Difference in average coverage between MOSA+LLM-ir and MOSA over time



(a) $MOSA + LLM - tl\_5$ **(b)** $MOSA + LLM - tl\_50$ **(c)** $MOSA + LLM - tl\_800$ **(d)** $MOSA + LLM - tldm\_5$

Figure 6: Difference in average coverage between MOSA+LLM-tl and MOSA over time



(a) $MOSA + LLM - tr\_005$ **(b)** $MOSA + LLM - tr\_5$ **(c)** $MOSA + LLM - tr\_10$ **(d)** $MOSA + LLM - trdm\_10$

Figure 7: Difference in average coverage between MOSA+LLM-tr and MOSA over time

most significant when LLM intervention occurs in the initializing population stage.

As for the case analysis in which LLM intervention failed to increase SBTG coverage but instead caused it to decrease, we considered the impact of changes in LLM intervention manners on this phenomenon and put the analysis in the next section.

4.2 RQ2: Manner of Intervention

In RQ2, we focus on the effect of LLM intervention in different manners at the same stage and different parameters in the same manner on final coverage and real-time coverage.

4.2.1 *Final coverage.*

Considering the results of the initializing population stage in Figure 2, it can be seen that $irdm\_10$ has the best overall effectiveness. The improvement effects of ir_05, ir_5, and ir_10 are weakened and the reduction effects are strengthened in turn.

If dynamic monitoring is not considered, it is better to apply LLM with a relatively small probability (between 0 and 0.1, such as 0.05). If LLM intervenes with an excessive probability, the effect may not be good.

> **Finding 4:** At the stage of initializing population, the best way to improve the average final coverage is to apply LLMs and perform dynamic monitoring.

Considering the results of the test coverage plateau stage in Figure 3, in general, the maximum stagnation length is best when it is in a suitable middle range, such as tl_50, which has both high test coverage improvement capability and low test coverage degradation capability. In addition, the improvement effect of $tldm\_5$ is not much different from tl_5, but the number of modules with reduced coverage has decreased.

After investigating the results, we find that when the maximum stagnation length is too small, such as 5, LLMs can easily interrupt the crossover and mutation of MOSA during execution, resulting in the core of the search algorithm, the search process, being blocked. Therefore the coverage is not significantly improved and it is easy to be lower than MOSA.

When the maximum stagnation length is too large, it is easy to know that the number of LLM interventions will be less, resulting in a relatively weaker coverage improvement effect.

When the maximum stagnation length takes a suitable intermediate value (such as 50 to 200), the intervention of LLMs can maximize the improvement of coverage. At this time, LLMs have enough intervention time to play a full role and do not affect the effectiveness of MOSA itself.

> **Finding 5:** At the test coverage plateau stage, the effectiveness of LLM intervention on the final coverage increases first and then decreases with the increase of the maximum stagnation length.

Considering the results of the iterative updating population stage in Figure 4, tr_005 has the greatest improvement on MOSA compared with other groups in most cases and there is almost no reduction in the coverage of any module. As the intervention probability increases, the coverage is lower than MOSA in more and more cases, such as tr_5 and tr_10. The reason for this phenomenon is intuitive. The frequent intervention of the LLM not only occupies a large amount of time that originally belonged to search but also affects the crossover and mutation functions that MOSA could effectively generate test cases.

> **Finding 6:** At the iterative updating population stage, LLMs should be applied with a relatively low probability.

From the above results, we notice that dynamic monitoring is conducive to improving coverage or reducing the possibility of coverage reduction, especially as shown in Figure 2, $irdm\_10$ has the best average effect. Through case studies, we find that dynamic monitoring not only helps prevent LLMs from excessively occupying the search time of MOSA but also ensures that crossover, mutation, and other operations of MOSA can function normally. Taking tl_5 and $tldm\_5$ as examples, each query of the LLM in tl_5 is performed when the coverage remains unchanged for 5 consecutive iterations. At the same time, operators such as crossover and mutation will randomly change some test cases each time, and the average number of iterations required for them to explore the new input space is much greater than 5 times. Therefore, although tl_5 advances the intervention time of the LLM, the maximum pause time of 5 iterations limits the effectiveness of the search algorithm of MOSA in the entire execution process. $tldm\_5$ adds dynamic monitoring based on tl_5 so that more time is allocated to the search process when querying the LLM cannot obtain new valid test cases, leading to a greater possibility of obtaining effective test cases through crossover and mutation within the maximum stagnation length limit. That is, the LLM gives in when its effectiveness is not good and then MOSA is used to improve the coverage.

In general, when the intervention probability is set to a large value or the maximum stagnation length is set to a small value, LLM

intervention fails to improve the coverage. We analyzed some cases and found that this situation was mainly caused by the failure of LLM intervention to generate valid test cases and the excessive time spent querying the LLM. Especially, when LLM intervention was not dynamically monitored and the frequency was high, the LLM frequently interrupted crossover, mutation, and other operations of MOSA, which took up too much time, resulting in a lower coverage than MOSA.

Finding 7: Regardless of the stage, dynamic monitoring effectively balances the time allocated to LLMs and SBGT in the test case generation process. It allows the LLMs to contribute without encroaching on too much time of SBGT or disrupting its normal execution.

4.2.2 Real-time coverage.

From Figure 5, Figure 6, and Figure 7, we find that when the LLM intervenes in the initial generation stage, regardless of the probability, the moments when the coverage is improved and when the coverage is reduced are mostly at the beginning. For *irdm*_10, the probability of coverage increasing or decreasing at the beginning is relatively high. When the LLM intervenes during the iterative updating population stage, the coverage is also improved immediately after the LLM begins to intervene. When the LLM intervenes during coverage stagnation, as the maximum stagnation length increases, the time when the coverage is improved will also be delayed.

Finding 8: The earlier the LLM intervenes, the earlier the coverage may be improved.

Answer to RQ2. A reasonable intervention frequency is crucial for LLMs to have a positive effect: changing the initial intervention probability at the initializing population stage or iterative updating population stage, dynamic monitoring for the whole process, or controlling the frequency of LLM intervention in the coverage plateau stage through a reasonable maximum stagnation length.

5 DISCUSSION

In this section, we discuss some implications of applying LLMs to SBTG and threats to validity of our study.

5.1 Implications

5.1.1 Implications for Researchers.

In our research, the basic idea of making LLMs interact with SBTG is to use LLMs to generate test cases and insert these test cases into a certain generation of test case population in SBTG to play a role. Specifically, the intervention of LLMs can improve code coverage in two aspects. First, *the test cases generated by LLMs can directly explore the input space that SBTG cannot search*, and the generated test cases themselves can increase the code coverage. Second, *starting from the test cases generated by LLMs, more input spaces that SBTG cannot search are explored through crossover, mutation, and other operators of SBTG*. We manually analyzed the Python modules on which the intervention of Code Llama had a significant impact and most of them have the following characteristics. The structure of the parameters is relatively simple, with no nesting or less nesting, such as dictionaries, lists, sets, etc. The types of parameters are also relatively simple, generally int, float, string, etc. Although it is difficult for MOSA to randomly generate parameters with real meaning, LLMs can generate expected parameters based on their understanding.

Future research may focus on improving the ability of LLMs to generate effective test cases, such as correctly constructing complex parameters. Another direction is applying LLMs to more processes of SBTG, such as allowing LLMs to help achieve test case mutations.

5.1.2 Implications for Developers.

Use LLMs as early as possible. The earlier LLMs are used, the faster they can achieve a high test coverage. Based on the experimental data and analysis in 4.1 and 4.2, developers can set parameters at various stages to allow LLMs to intervene in MOSA as early as possible. Among them, the most likely way to improve early coverage and the most obvious effect is to let LLMs intervene during the generation of the initial population, set the initial intervention probability of LLMs to 1.0, and dynamically monitor it.

Balance the use of LLMs and SBTG. SBGT and LLMs serve different roles in the test case generation process, and their complementary use can more efficiently improve test coverage. As an effective method for generating test cases, SBGT should not be excessively interrupted or have its allocated time encroached upon. At the same time, LLMs have shown their excellence in generating specific types of test cases. Therefore, applying dynamic monitoring and adjusting the frequency of LLM intervention accordingly is an effective strategy.

5.2 Threats to Validity

Internal Validity. Our construction of query prompts for LLMs is relatively basic. More sophisticated prompt engineering can provide LLMs with more effective prompts, so that LLMs can generate effective test cases for more complex and diverse functions/methods, bringing more possibilities for the improvement of SBTG. After getting the results using Code Llama, we need to post-process them to obtain test cases consistent with the MOSA format. In the experiments in this paper, we used Pynguin as the framework of MOSA, so we rewrote the output of Code Llama through a series of operations to make it conform to the format of Pynguin. Although we have done our best to ensure that the output of Code Llama could be successfully used in Pynguin, there might be some test code that could be modified or adjusted to be used.

External Validity. Due to limited time, we filtered some projects from the original dataset. Although we conducted preliminary experiments to confirm that our selection is reasonable, experiments can still be conducted on larger datasets in the future. We used MOSA and Code Llama as representatives of SBTG and LLMs respectively. Although they are representative methods in their respective fields, more SBTG algorithms and LLMs can be used in the future to test whether the research results of this paper are still valid. We only used one hyperparameter setting of Code Llama. Although this was determined based on the experimental results of CODAMOSA, future studies can set different hyperparameter experiments to verify the conclusions of this paper. We only focused on Python projects in this study, but whether the conclusions can be generalized to other programming languages, such as Java, still needs further exploration.

Construct Validity. We used code coverage as the metric for search algorithms to measure test cases. The impact of applying LLMs to SBTG on other metrics is also worth discussing.

6 RELATED WORK

In this section, we introduce some related work of our study, including search-based test generation and large language models for testing.

6.1 Search-Based Test Generation

The earliest application of search algorithms in test data generation can be traced back to the 1970s [30]. With the rise of meta-heuristic search algorithms, such as genetic algorithms [6, 18, 22], tabu search [15], simulated annealing [36], and particle swarm optimization [21], they began to be applied to software testing [3, 17, 29], mainly to optimize the selection and generation of test cases to improve test coverage and efficiency.

Considering the limitation of the search algorithm, some studies have begun to explore the combination of SBTG and other technologies. K.Inkumsah et al. [19] implemented a new framework Evacon that integrates evolutionary testing and symbolic execution. M.Harman et al. [5] proposed a new algorithm that combines symbolic information with dynamic analysis for constructing fitness functions. J.Malburg et al. [28] combined metaheuristic-based search technology and constraint-based testing technology and the method had better experimental results than both.

As existing SBTG technologies often focused on coverage but ignored the cost in test case size, M. Harman et al. [16] proposed three improved algorithms that can improve test coverage without affecting test coverage and reduce the number of test cases generated by search-based testing. Arcuri and Yao [4], as well as Baresi et al. [8], used a single sequence of function calls to maximize the number of branches covered while minimizing the length of test cases. In the work of Baudry et al. [9], a search algorithm on mutation analysis was used to optimize the test suite. G.Fraser and A.Arcuri [14] introduced the concept of using search-based techniques to generate entire test suites, aiming to simultaneously optimize multiple conflicting goals such as code coverage and test suite size.

However, despite these positive advances of various methods, SBTG still faces the ongoing challenge of search stagnation [1, 2], which impacts its effectiveness and efficiency. Besides, SBTG techniques also face severe time-consuming challenges due to complex search space or required execution and iteration. These gaps are what our study intends to fill through investigation.

6.2 Large Language Models for Testing

Based on the ability of Large Language Models (LLMs) to understand and generate natural language and code, researchers began to explore the potential of LLMs to generate test cases and various methods have been adopted to guide LLMs for various applications.

Some studies use user intentions as prompts. For example, TICODER [23] and CODET [10] use Codex to generate corresponding codes and test cases based on problems described in natural language given by users. Similar studies have also been conducted on ChatGPT to generate tests [25, 26, 33, 39, 40].

Some methods provide more diverse code information to LLMs as prompts. For example, Bareiß et al. [7] embed contextual information into prompts to guide LLMs, including the function definitions under test, examples of different functions and their related tests, and a list of related helper function signatures. TESTPILOT [32] is an adaptive technique for generating unit tests using LLMs. In addition to the source code and documentation of the function, it can also use the generated failed test cases and their execution information as prompts to LLMs. Tufano et al. proposed AthenaTest [35], which relies on heuristics to identify the focus classes and methods under test and expresses the task of generating unit tests as focusing methods. MuTAP [11] uses surviving mutants to enhance prompts and generate test cases, improving the effectiveness of test cases generated by LLMs.

Additionally, there are also some studies that combine LLMs with traditional automatic test generation techniques. Considering that traditional techniques often struggle to generate meaningful or valid assertions, ATLAS [37] utilizes neural machine translation (NMT) to generate unit test assertion statements for test methods. Inspired by assertion generation, TOGA [12] obtains both exceptions and assertions based on the context reasoning of the focus method to test Oracle. Stallenberg et al. [34] proposed an unsupervised type JavaScript test generation technique that consists of three stages: performing static analysis to derive relationships between program elements, applying probabilistic type inference to build models, and using the models in search-based software testing techniques.

LIBRO [20] proposed a feasible solution for generating test cases that can reproduce the errors in the bug report. It takes the given bug report as a prompt to let LLMs generate the expected test cases.

The results of the above exploratory studies show that LLMs have great potential and good scalability and adaptability in software testing, which inspired our research.

7 CONCLUSION

Considering that existing studies lack comprehensive and in-depth research on the intervention of LLMs in SBTG and the practical needs of developers, it is important to analyze various strategies for the intervention of LLMs in SBTG. In this study, we proposed 6 intervention strategies and conducted experiments on 486 Python modules. The experimental results show that involving LLMs in various opportunities has the potential to improve SBTG test coverage and efficiency. However, if we want the effect of LLM intervention to be as good as possible, the manner of LLM intervention needs to be controlled. Specifically, the frequency of LLM intervention should be within a reasonable range. Too high a frequency of LLM intervention will not only fail to improve SBTG but also reduce the effect of SBTG. In general, the intervention strategy with the most significant and stable average effect is to intervene during the generation of the initial population, with a larger intervention probability and dynamic monitoring.

ACKNOWLEDGMENTS

This work was partially supported by the National Natural Science Foundation of China (62272221).

REFERENCES

[1] Nasser Albunian, Gordon Fraser, and Dirk Sudholt. 2020. Causes and effects of fitness landscapes in unit test generation. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*. 1204–1212.

[2] Aldeida Aleti, Irene Moser, and Lars Grunske. 2017. Analysing the fitness landscape of search-based software testing problems. *Automated Software Engineering* 24 (2017), 603–621.

[3] Shaukat Ali, Lionel C Briand, Hadi Hemmati, and Rajwinder Kaur Panesar-Walawege. 2009. A systematic review of the application and empirical investigation of search-based test case generation. *IEEE Transactions on Software Engineering* 36, 6 (2009), 742–762.

[4] Andrea Arcuri and Xin Yao. 2008. Search based software testing of object-oriented containers. *Information Sciences* 178, 15 (2008), 3075–3095.

[5] Arthur Baars, Mark Harman, Youssef Hassoun, Kiran Lakhotia, Phil McMinn, Paolo Tonella, and Tanja Vos. 2011. Symbolic search-based testing. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 53–62.

[6] Thomas Back. 1996. *Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms*. Oxford university press.

[7] Patrick Bareiß, Beatriz Souza, Marcelo d'Amorim, and Michael Pradel. 2022. Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code. *arXiv preprint arXiv:2206.01335* (2022).

[8] Luciano Baresi, Pier Luca Lanzi, and Matteo Miraz. 2010. Testful: An evolutionary test approach for java. In *2010 Third International Conference on Software Testing, Verification and Validation*. IEEE, 185–194.

[9] Benoit Baudry, Franck Fleurey, J-M Jézéquel, and Yves Le Traon. 2005. Automatic test case optimization: A bacteriologic algorithm. *IEEE software* 22, 2 (2005), 76–82.

[10] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397* (2022).

[11] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* (2024), 107468.

[12] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.

[13] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.

[14] Gordon Fraser and Andrea Arcuri. 2012. Whole test suite generation. *IEEE Transactions on Software Engineering* 39, 2 (2012), 276–291.

[15] Fred Glover. 1990. Tabu search: A tutorial. *Interfaces* 20, 4 (1990), 74–94.

[16] Mark Harman, Sung Gon Kim, Kiran Lakhotia, Phil McMinn, and Shin Yoo. 2010. Optimizing for the number of tests generated in search based test data generation with an application to the oracle cost problem. In *2010 Third International Conference on Software Testing, Verification, and Validation Workshops*. IEEE, 182–191.

[17] Mark Harman, S Afshin Mansouri, and Yuanyuan Zhang. 2009. Search based software engineering: A comprehensive analysis and review of trends techniques and applications. (2009).

[18] John H Holland. 1992. *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press.

[19] Kobi Inkumsah and Tao Xie. 2008. Improving structural testing of object-oriented programs via integrating evolutionary testing and symbolic execution. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 297–306.

[20] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.

[21] James Kennedy and Russell Eberhart. 1995. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, Vol. 4. ieee, 1942–1948.

[22] John R Koza. 1994. Genetic programming as a means for programming computers by natural selection. *Statistics and computing* 4 (1994), 87–112.

[23] Shuvendu K Lahiri, Aaditya Naik, Georgios Sakkas, Piali Choudhury, Curtis von Veh, Madanlal Musuvathi, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2022. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950* (2022).

[24] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931.

[25] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, and Shing-Chi Cheung. 2023. Finding failure-inducing test cases with chatgpt. *arXiv preprint arXiv:2304.11686* (2023).

[26] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Chatting with gpt-3 for zero-shot human-like mobile automated gui testing. *arXiv preprint arXiv:2305.09434* (2023).

[27] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 168–172.

[28] Jan Malburg and Gordon Fraser. 2011. Combining search-based and constraint-based testing. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 436–439.

[29] Phil McMinn. 2004. Search-based software test data generation: a survey. *Software testing, Verification and reliability* 14, 2 (2004), 105–156.

[30] Webb Miller and David L. Spooner. 1976. Automatic generation of floating-point test data. *IEEE Transactions on Software Engineering* 3 (1976), 223–226.

[31] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2017. Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering* 44, 2 (2017), 122–158.

[32] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* (2023).

[33] Mohammed Latif Siddiq, Joanna Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2023. Exploring the effectiveness of large language models in generating unit tests. *arXiv preprint arXiv:2305.00418* (2023).

[34] Dimitri Stallenberg, Mitchell Olsthoorn, and Annibale Panichella. 2022. Guess what: Test case generation for Javascript with unsupervised probabilistic type inference. In *International Symposium on Search Based Software Engineering*. Springer, 67–82.

[35] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).

[36] PT Van Larrhoven and EH Aarts. 1988. Simulated annealing: theory and practice.

[37] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1398–1409.

[38] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, et al. 2020. Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 1556–1560.

[39] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: a ChatGPT-based automated unit test generation tool. *arXiv preprint arXiv:2305.04764* (2023).

[40] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? evaluating and improving chatgpt for unit test generation. *arXiv preprint arXiv:2305.04207* (2023).